

Progetto Bomberman

Antonio Luca Pirrone, Manuel Sgarbi e Vittorio Biagini

Febbraio 2026

1 Descrizione del progetto

Il Progetto è una versione in caratteri ascii del primo Bomberman per NES. Nella nostra versione è presente un menu animato, una classifica che elenca i migliori risultati, salvati su un file di testo e ovviamente il gioco stesso, che come nell'originale presenta livelli completamente esplorabili, nemici e l'abilità di piazzare bombe, oltre che nuove aggiunte come vari potenziamenti ottenibili dal giocatore e diversi nemici con abilità diverse. Per utilizzare il gioco si usano le freccette della tastiera oppure WASD, assieme al tasto INVIO, ESC e P e E per funzioni specifiche in partita.

2 Suddivisione parti

Antonio Luca Pirrone:

In questo progetto mi sono occupato della generazione e della gestione della **mappa** e dei **livelli**, dell' **interfaccia utente**, del **cronometro** in gioco, dell'implementazione degli **Item** e della **struttura generale dell'input loop** e del movimento del giocatore, poi completato e raffinato dal mio collega Manuel Sgarbi.

2.1 Mappa e livelli

La mappa è comprensiva di **5 livelli** implementati tramite matrici di caratteri 23 x 43, collagate fra loro tramite doppia linked list. La generazione dei livelli e la loro traduzione sullo schermo sono processi distinti, implementati tramite funzioni separate che lavorano consequenzialmente e modularmente. Quando un livello viene completato (ovvero quando tutti i nemici presenti sono sconfitti) questo diventa inaccessibile (tolto dalla lista) una volta abbandonato, è tuttavia possibile viaggiare liberamente da e verso qualsiasi livello ancora da completare. Un aspetto importante della mappa è che quest'ultima è **simbolica**, ovvero rappresenta gli elementi al suo interno tramite caratteri diversi da quelli mostrati a schermo, che svolgono una semplice **funzione logica**. Lo schermo è una window di dimensioni 25 x 45, che mostra la mappa (con la corretta astrazione grafica) e l'interfaccia utente.

```

17 void Level::printStats(Map* level, WINDOW* screen) {
18     init_pair(4, COLOR_YELLOW, COLOR_BLACK);
19     init_pair(5, COLOR_GREEN, COLOR_BLACK);
20
21     for (int y=0; y<3; y++)
22         for (int x=0; x<4; x++)
23             switch (level->level[y][x]) {
24
25                 case 'F':
26                     mvwaddch(screen, y+1, x+1, ' ' | A_REVERSE);
27                     break;
28
29                 case '+':
30                     mvwaddch(screen, y+1, x+1, ACS_CROSS);
31                     break;
32
33                 case 'B':
34                     wattron(screen, COLOR_PAIR(4));
35                     mvwaddch(screen, y+1, x+1, '2');
36                     wattroff(screen, COLOR_PAIR(4));
37                     break;
38
39                 case 'S':
40                     wattron(screen, COLOR_PAIR(4));
41                     mvwaddch(screen, y+1, x+1, 'S');
42                     wattroff(screen, COLOR_PAIR(4));
43                     break;
44
45                 case 'T':
46                     wattron(screen, COLOR_PAIR(4));
47                     mvwaddch(screen, y+1, x+1, 'T');
48                     wattroff(screen, COLOR_PAIR(4));
49                     break;
50
51                 case 'U':
52                     wattron(screen, COLOR_PAIR(4));
53                     mvwaddch(screen, y+1, x+1, 'U');
54                     wattroff(screen, COLOR_PAIR(4));
55                     break;
56
57                 default:
58                     mvwprintw(screen, y+1, x+1, "%d", level->level[y][x]);
59                     break;
60
61             }
62     wattron(screen, COLOR_PAIR(3));
63     item_hide_items(level, screen);
64     wattroff(screen, COLOR_PAIR(3));
65     box(screen, 0, 0);
66 }

```

Ecco ad esempio la funzione "printStats", che svolge il compito di interpretare la mappa e stamparla sulla window che viene mostrata a schermo.

2.2 Interfaccia utente

L'interfaccia utente è stata implementata perlopiù come messaggi (punti, cronometro, vite, livello corrente, potenziamenti raccolti o attivi) mostrati sulla cornice della finestra dello schermo, ma è presente anche una **finestra di pausa** da cui è possibile consultare i comandi, accessibile in qualsiasi momento durante la partita premendo P.

2.3 Item

Gli item sono degli **oggetti raccogliabili** sulla mappa, rilasciati come loot dai nemici una volta uccisi. Nella implementazione degli item è risultata particolarmente utile la **logica dell'astrazione** della mappa: sulla mappa gli item sono memorizzati come lettere minuscole, ognuna delle quali corrisponde a un potenziamento specifico, ma esiste una funzione chiamata **"hideitems"** all'interno della classe **ItemList** che stampa ogni item su schermo come un "?", in questo modo è preservata una logica pulita pur dando un'**illusione di "casualità"** al giocatore. Alcuni item sono **a tempo limitato (10s)**, questi non sono cumulabili, e raccogliere un item a tempo durante l'effetto attivo di un altro lo sovrascriverà. L'effetto attivo al momento è mostrato nell'angolo in alto a destra dello schermo. Esistono anche item dall'effetto permanente.

2.4 Input loop

L'implementazione dell'**input loop** iniziale è stata messa a punto da me (nonostante riguardasse il movimento del player e fosse dunque competenza del mio

collega Manuel Sgarbi) per semplici motivi di **debug** nelle fasi iniziali della scrittura del codice di **"Levels.cpp"**. Le parti che avevo implementato io riguardano il movimento di base e l'apertura del menù di pausa. Il resto è stato aggiunto dal mio collega ed è stato trattato più avanti in questa relazione da egli stesso.

Manuel Sgarbi:

In questo progetto mi sono occupato principalmente dell'implementazione del **giocatore**, dei **nemici** e di tutte le meccaniche di gioco direttamente collegate a queste entità, come il movimento, le collisioni, la gestione dei danni, il sistema di punteggio e l'interazione con le bombe.

2.5 Giocatore

Il giocatore è rappresentato da un singolo carattere ASCII e può muoversi all'interno della mappa tramite input da tastiera (WASD o frecce direzionali). Il movimento è vincolato dalla presenza di muri solidi e distruttibili, mentre le collisioni con i nemici vengono rilevate prima dell'effettivo spostamento, evitando situazioni incoerenti durante il gameplay.

Il giocatore può inoltre piazzare bombe che esplodono dopo un intervallo di tempo prefissato. Le esplosioni possono danneggiare sia il giocatore sia i nemici, rendendo necessario un utilizzo attento e strategico delle bombe. Per evitare la perdita di più vite in un tempo troppo breve, è stato implementato un sistema di **invincibilità temporanea**, accompagnato da un effetto visivo di blink che segnala chiaramente lo stato al giocatore.

2.6 Nemici

Sono stati implementati diversi tipi di nemici, caratterizzati da comportamenti e velocità differenti: nemici lenti, nemici veloci, nemici più resistenti e un nemico speciale in grado di piazzare bombe autonomamente.

Il movimento dei nemici è regolato da timer indipendenti basati sul tempo reale, così da garantire un comportamento coerente e uniforme anche in un contesto cross-platform. Ogni nemico si muove in modo casuale, rispettando i limiti della mappa e gli ostacoli presenti. Le collisioni con il giocatore comportano la perdita di una vita, salvo nei casi in cui sia attivo un effetto di invincibilità.

2.7 Danni, punteggio e bombe

Le bombe generano un'esplosione che si propaga nelle quattro direzioni cardinali fino a incontrare un muro solido o a esaurire la potenza del raggio. Quando un nemico viene colpito, può essere eliminato oppure trasformato, nel caso dei nemici più resistenti. In ogni caso viene incrementato il punteggio del giocatore in base al tipo di nemico colpito e può essere generato un oggetto raccogliabile.

È stata posta particolare attenzione alla gestione temporale delle esplosioni, in modo da evitare che una singola bomba possa causare danni o assegnazioni

di punteggio multiple nello stesso istante.

Vittorio Biagini:

In questo progetto mi sono occupato della creazione e gestione dei menu di gioco, tra cui il **menu principale**, la schermata di visualizzazione della **classifica**, la **gestione dei file** di salvataggio dei punteggi per la classifica e la parte grafica della schermata di **pausa**.

2.8 Menu principale

Il menu principale gestisce l'indirizzamento del giocatore nelle vari parti del progetto. Dopo aver sviluppato la funzione di base del menu mi sono potuto divertire nell'abbellirlo e renderlo il più funzionale possibile. In particolare, il menu presenta dei sottotitoli che ciclan periodicamente grazie a un metodo di calcolo del tempo e il titolo del gioco rappresentato in "ascii art" animato da una pulsazione e, in caso si selezion "Nuova Partita" animato dall'esplosione di 3 bombe generate casualmente nella zona del titolo, che esplodendo lo distruggono. La difficoltà principale è stata l'implementazione dell'animazione di distruzione del titolo, gestita salvando il titolo in una matrice (così da salvare le sue modifiche, mantenendolo distrutto) e funzioni di calcolo del tempo e gestione del piazzamento ed esplosione per le bombe.

2.9 Classifica

La schermata classifica prende gli elementi salvati nel file Leaderboard.txt per stamparli in ordine decrescente. La classifica è scorribile così da poter visualizzare tutti gli elementi e presenta una scritta in "ascii art" con animazione pulsante. La difficoltà principale è stata la logica dello scorrimento e in generale la creazione di una schermata "pulita" con nCurses.

2.10 Gestione del file

Nelle classi Leaderboard.cpp e Score.cpp è stata gestita la scrittura, lettura e ordinamento per la stampa dei vari elementi del file di testo adibito al salvataggio dei punteggi delle partite

2.11 Indirizzamento del menu

La classe Game.cpp gestisce la selezione dell'input utente del menu, chiamando i metodi "run" delle classi Leaderboard e Levels. Inoltre fa da supporto a Menu.cpp, passandogli gli elementi del menu da passare a schermo.

3 Utilizzo LLM

Antonio Luca Pirrone:

Personalmente ho usato ChatGPT come ”consigliere”, sfruttandolo per ottimizzare il codice, velocizzare il debug e in certi casi anche ricevere delle valutazioni e dei commenti sul codice scritto. E’ capitato che notasse bug e proponesse soluzioni, alcune delle quali sono state adottate anche nel codice finale.

```
//cambio di livello
if (current_level->level[p.getY()][p.getX()] == '<' &&
!levelCleared[current_level->previous->index]) {
    current_level = current_level->previous;
    lvl = current_level->index;
    p.setPosition(21, 41);
    if (levelCleared[current_level->next->index] == true) {
        map* tmp = current_level->next;
        current_level->next = current_level->next->next;
        delete tmp;
        time_left += 60;
    }
}
else if (current_level->level[p.getY()][p.getX()] == '>' &&
!levelCleared[current_level->next->index]) {
    current_level = current_level->next;
    lvl = current_level->index;
    p.setPosition(1, 1);
    if (levelCleared[current_level->previous->index] == true) {
        map* tmp = current_level->previous;
        current_level->previous = current_level->previous->previous;
        delete tmp;
        time_left += 60;
    }
}
```

```
c
if (current_level->level[p.getY()][p.getX()] == '<' && !levelCleared[current_level->previous->index]) {
    current_level = current_level->previous;
    lvl = current_level->index;
    p.setPosition(21, 41);
    if (levelCleared[current_level->next->index] == true) {
        map* tmp = current_level->next;
        current_level->next = current_level->next->next;
        delete tmp;
        time_left += 60;
    }
}
else if (current_level->level[p.getY()][p.getX()] == '>' && !levelCleared[current_level->next->index]) {
    current_level = current_level->next;
    lvl = current_level->index;
    p.setPosition(1, 1);
    if (levelCleared[current_level->previous->index] == true) {
        map* tmp = current_level->previous;
        current_level->previous = current_level->previous->previous;
        delete tmp;
        time_left += 60;
    }
}
```

Correzione bug proposta da ChatGPT

Segmento di codice dato in analisi a ChatGPT

Manuel Sgarbi:

Durante lo sviluppo del progetto è stato utilizzato un **Large Language Model (LLM)** come strumento di supporto alla programmazione. L’LLM è stato impiegato principalmente per individuare bug logici, migliorare la struttura del codice e chiarire problematiche legate alla gestione del tempo, delle collisioni e delle esplosioni.

Il suo utilizzo ha contribuito a una migliore comprensione delle meccaniche di gioco e a una progettazione più solida del sistema, mantenendo comunque un approccio critico alle soluzioni proposte, che sono state integrate solo dopo verifica e adattamento al contesto specifico del progetto.

3.1 Esempi di utilizzo dell’LLM

Il Large Language Model è stato utilizzato esclusivamente come strumento di supporto allo sviluppo. Alcuni esempi di richieste effettuate includono:

- analisi di bug legati alla gestione del tempo e delle animazioni;
- suggerimenti per separare correttamente la fase di aggiornamento della logica di gioco dalla fase di rendering;

- verifica della correttezza delle collisioni tra giocatore, nemici ed esplosioni;
- miglioramento della leggibilità del codice e riduzione di condizioni ridondanti;
- supporto nell'apprendimento di linguaggi e strumenti nuovi, come L^AT_EX.

Le risposte fornite dall'LLM non sono mai state applicate in modo automatico, ma sempre valutate con attenzione e adattate alle esigenze specifiche del progetto e del contesto di sviluppo.

Vittorio Biagini:

Il mio utilizzo degli LLM è stato fatto attraverso l'uso di ChatGPT, specialmente per il debugging oltre che per un'iniziale spiegazione di come scaricare la libreria nCurses su Windows. L'LLM si è risultato particolarmente utile nella risoluzione di bug relativi alla stampa del menu e l'esplosione del titolo su terminali Linux, dei quali non sono particolarmente familiare, oltre che la risoluzione di bug relativi alla lettura dal file di testo Leaderboard e un sostegno generale anche per l'apprendimento e utilizzo delle funzioni di nCurses.

```
double Menu::nowSec() {
    return (double)clock() / CLOCKS_PER_SEC;
}
```

Segmento di codice dato in analisi a ChatGPT

clock() misura CPU time, non "tempo reale". In un busy-loop su Linux cresce in modo diverso e ti altera animazioni/pulse/temporizzazioni.

Sostituisci nowSec() con un clock monotono:

```
Cpp Copia codice
#include <chrono>

double Menu::nowSec() {
    using namespace std::chrono;
    static const auto t0 = steady_clock::now();
    return duration<double>(steady_clock::now() - t0).count();
}
```

Correzione proposta da ChatGPT